

Instituto Superior de Engenharia de Lisboa

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Relatório Intermédio de Projeto

API de comunicação para *smart tool cabinets*

Unidade curricular	Projeto - LEIRT
Aluno	Ricardo Nobre (48713)
Orientadores	Tiago Dias e Carlos Gonçalves
Ano letivo	2025/2026

Lisboa, maio de 2026

Índice

1.	Introdução	1
2.	Enquadramento e delimitação do problema.....	2
3.	Requisitos.....	3
4.	Abordagem técnica da solução.....	4
5.	Especificação da API.....	5
6.	Modelo de dados	7
7.	Estado atual do desenvolvimento	8
8.	Calendarização atualizada e ponto de situação	9
9.	Próximos passos até ao relatório final.....	10
10.	Conclusão	11

1. Introdução

Este relatório intermédio dá continuidade ao relatório inicial do projeto “API de comunicação para *smart tool cabinets*”, desenvolvido no âmbito da unidade curricular Projeto da Licenciatura em Engenharia Informática, Redes e Telecomunicações. O objetivo geral mantém-se centrado na definição de uma API central para *smart tool cabinets*, capaz de suportar a comunicação entre os armários inteligentes e o sistema central, bem como o registo consistente das operações relevantes.

Desde o relatório inicial, o trabalho avançou sobretudo na delimitação do problema, na organização dos requisitos, na especificação da API e na preparação da documentação técnica de apoio. A abordagem atual é *API-first*, em que a interface de comunicação e a respetiva especificação técnica são os principais objetivos do trabalho a desenvolver na fase seguinte.

Este relatório descreve os requisitos que orientam a API, sintetiza a abordagem técnica adotada, apresenta a especificação dos *endpoints* principais, descreve o modelo de dados essencial e identifica o estado atual do desenvolvimento. Por fim, é atualizada a calendarização do trabalho e são indicados os próximos passos até ao relatório final.

2. Enquadramento e delimitação do problema

O projeto enquadra-se no desenvolvimento de um sistema central para gestão e monitorização de *smart tool cabinets* utilizados em contextos onde é necessário acompanhar a utilização de ferramentas e manter rastreabilidade das operações realizadas. A componente tratada neste trabalho incide sobre a comunicação entre os armários e o sistema central, através de uma API que permita autenticar armários e operadores, receber leituras de inventário, registar eventos e expor informação sobre o estado de custódia das ferramentas.

No relatório inicial, o problema encontrava-se ainda descrito com maior proximidade ao conceito de sessão de armário e à validação do fecho físico com eventual recusa em caso de ausência de ferramentas. Com a evolução do estudo e com o feedback recebido nas reuniões de acompanhamento, essa visão foi ajustada: o armário fecha sempre fisicamente, cabendo à API registar os factos operacionais, calcular o resultado do processamento e disponibilizar informação para posterior análise ou resolução.

O foco principal é a API de comunicação, a especificação do *backend* que a deverá suportar, o modelo de dados essencial e a validação conceptual do fluxo principal. Não se pretende modelar todos os processos internos da empresa, construir uma interface de gestão completa, desenvolver análise histórica avançada ou representar variações de *hardware* que não sejam necessárias para a especificação da API.

Incluído nesta fase	Possível evolução futura
Definição da API principal de comunicação	Painéis de visualização e análise histórica
Modelo de <i>backend</i> e persistência essencial	Extensões de <i>hardware</i> com múltiplas gavetas
Fluxo de acesso, <i>snapshots</i> e <i>close</i>	Integrações externas avançadas
Custódia por operador e pendências	Reforço avançado de segurança e auditoria

3. Requisitos

Os requisitos definidos resultam do funcionamento esperado da API e da necessidade de manter o projeto tecnicamente focado. Procurou-se estabelecer um conjunto de requisitos suficientemente claro para orientar a especificação e a futura implementação, evitando complexidade que não seja necessária para a entrega intermédia.

3.1. Requisitos funcionais

ID	Requisito	Descrição
RF1	Autenticação do armário	A API deve permitir que o armário se autentique antes de enviar pedidos operacionais.
RF2	Autenticação do operador	A API deve validar a identidade do operador por <i>PIN</i> ou <i>NFC</i> no contexto do armário.
RF3	Abertura de acesso ao armário	A API deve criar um <i>CabinetAccess</i> para representar uma interação local curta com o armário.
RF4	Registo de <i>snapshots</i>	A API deve receber leituras <i>BEFORE</i> e <i>AFTER</i> para permitir calcular alterações no estado das ferramentas.
RF5	Fecho com resultado operacional	O <i>endpoint</i> de <i>close</i> deve encerrar o acesso e devolver um <i>operationalResult</i> , sem bloquear o fecho físico do armário.
RF6	Atualização da custódia	O delta entre <i>snapshots</i> deve criar ou encerrar <i>ToolAssignments</i> , representando retirada ou devolução de ferramentas.
RF7	Verificação de fim do dia	A API deve indicar se o operador mantém ferramentas pendentes no final do período de trabalho.
RF8	Resolução de supervisor	Quando existirem pendências, a API deve permitir o registo de <i>reasonCode</i> , <i>reportText</i> e <i>allowExit</i> .
RF9	Rastreabilidade mínima	A API deve manter registos suficientes para reconstruir operações relevantes.

3.2. Requisitos não funcionais

ID	Requisito	Descrição
RNF1	Clareza do contrato	Os <i>endpoints</i> , <i>payloads</i> e <i>responses</i> devem estar documentados de forma consistente em <i>OpenAPI</i> .
RNF2	Consistência dos dados	Os identificadores das ferramentas e os estados de custódia devem evitar ambiguidades.
RNF3	Rastreabilidade	As operações relevantes devem poder ser reconstruídas através de eventos, atribuições e resoluções.
RNF4	Segurança base	Os <i>endpoints</i> devem distinguir pedidos de armário, operador, supervisor e administração.
RNF5	Testabilidade	O fluxo principal deve ser demonstrável através de <i>requests</i> HTTP e <i>responses</i> verificáveis.

4. Abordagem técnica da solução

A solução proposta organiza-se em torno de uma API central, a desenvolver em Java e Spring Boot. O armário inteligente atua como cliente da API, enviando pedidos relativos à sua autenticação, à autenticação do operador, à abertura do acesso operacional, aos *snapshots* de inventário, aos eventos técnicos e ao fecho do acesso. O *backend* deverá processar esses pedidos, aplicar as validações principais e persistir o estado relevante na base de dados.

A organização atual separa os principais conceitos do domínio de forma mais precisa do que a proposta inicial. O conceito de CabinetAccess representa a interação local com o armário; ToolAssignment representa o estado de custódia da ferramenta por operador; SupervisorResolution formaliza uma decisão humana quando existem pendências; e AuditLog mantém rastreabilidade transversal das ações relevantes.

Componente	Papel no projeto
Armário inteligente	Cliente previsto da API; envia autenticação, <i>snapshots</i> , eventos e pedidos de <i>close</i> .
API/ <i>backend</i> previsto	Recebe <i>requests</i> , aplica validações, calcula o estado operacional e expõe <i>endpoints</i> de consulta.
Base de dados prevista	Deve persistir armários, utilizadores, ferramentas, acessos, <i>snapshots</i> , atribuições, resoluções e eventos.
<i>OpenAPI/Swagger</i>	Documenta o contrato da API e serve de referência técnica para <i>requests</i> e <i>responses</i> .
Auditoria	Deve registrar ações relevantes para rastreabilidade, sem substituir a decisão formal do supervisor.

5. Especificação da API

A API foi organizada por blocos funcionais, separando os pedidos realizados pelo armário, as consultas associadas ao operador, as decisões do supervisor e os *endpoints* administrativos mínimos necessários para preparação dos dados e futura demonstração.

O contrato técnico é descrito em *OpenAPI*, permitindo documentar *endpoints*, *payloads*, respostas, esquemas de segurança e estruturas de dados. No relatório, a especificação é apresentada de forma explicativa, destacando o objetivo de cada *endpoint* e a semântica operacional do fluxo principal.

Bloco	Endpoint	Objetivo
Device	POST /api/device/auth	Autenticar o armário antes de operar.
Device	POST /api/device/operator-auth	Validar o operador por <i>PIN</i> ou <i>NFC</i> .
Device	POST /api/device/cabinet-accesses	Criar um acesso operacional local ao armário.
Device	POST /api/device/cabinet-accesses/{id}/snapshots	Registrar <i>snapshot</i> BEFORE/AFTER para cálculo de delta.
Device	POST /api/device/cabinet-accesses/{id}/close	Fechar o acesso e devolver resultado operacional.
Device	POST /api/device/cabinet-events	Registrar eventos técnicos relevantes do armário.
Operator	GET /api/operators/{operatorId}/tool-assignments	Consultar o estado de custódia do operador.
Operator	GET /api/operators/{operatorId}/end-of-day-check	Verificar pendências no fim do dia.
Supervisor	POST /api/supervisor/resolutions	Registrar decisão formal do supervisor.
Supervisor	GET /api/supervisor/resolutions	Consultar resoluções registradas.
Admin	POST /api/admin/cabinets	Criar dados técnicos mínimos de armários para operação e demonstração.
Admin	POST /api/admin/tools	Criar dados técnicos mínimos de ferramentas para operação e demonstração.
Admin	POST /api/admin/users	Criar dados técnicos mínimos de utilizadores para operação e demonstração.

5.1. Semântica operacional do close

O *endpoint* de *close* não representa uma autorização para o armário fechar fisicamente. O armário fecha sempre. A finalidade deste *endpoint* é encerrar o CabinetAccess e devolver um resultado operacional calculado a partir do delta entre os *snapshots* BEFORE e AFTER.

Esta opção evita que a API seja responsável por resolver fisicamente situações de falta de ferramentas. A API comunica o estado operacional, atualiza a custódia e mantém rastreabilidade. Quando uma ferramenta permanece por devolver, essa situação pode ser consultada no fim do dia e, se necessário, formalizada por uma resolução de supervisor.

5.2. Tipos de pedidos e respostas previstos

Esta subsecção identifica os principais tipos de pedidos e respostas que representam o fluxo funcional da API: autenticação do armário, autenticação do operador, abertura de CabinetAccess, registo de *snapshots*, *close* e SupervisorResolution. A sua apresentação serve de base à implementação futura, permitindo verificar se os pedidos e as respostas previstos na especificação são suficientes para suportar o comportamento esperado do sistema.

Exemplo	Elemento demonstrado
Autenticação do armário	Autenticação prevista do armário e emissão de token temporário.
Autenticação do operador	Separação entre autenticação do operador e abertura do acesso.
<i>Snapshot</i> BEFORE/AFTER	Registo do estado observado do armário para cálculo de delta.
<i>Close</i>	Resposta esperada com <i>operationalResult</i> e contadores operacionais.
SupervisorResolution	Registo humano previsto de <i>reasonCode</i> , <i>reportText</i> e <i>allowExit</i> .

6. Modelo de dados

Este modelo de dados foi definido de modo a garantir que cada *endpoint* possui uma base persistente coerente e que os principais estados do fluxo possam ser consultados e auditados.

As entidades principais incluem Cabinet, AppUser, Tool, CabinetAccess, InventorySnapshot, ToolAssignment, SupervisorResolution, CabinetEvent e AuditLog. A entidade ToolAssignment é particularmente relevante, pois representa o estado de custódia da ferramenta por operador, permitindo distinguir ferramentas ativas, devolvidas, pendentes de revisão ou resolvidas.

Entidade	Função no modelo
cabinet	Representa o armário, identificado por code e credenciais técnicas.
app_user	Representa operadores, supervisores e administradores.
tool	Representa ferramentas <i>RFID</i> , com tag_code único, type_code e serial_number opcional.
cabinet_access	Representa a interação local curta entre operador e o armário.
inventory_snapshot	Guarda leituras BEFORE, AFTER ou EXTRA enviadas pelo armário.
tool_assignment	Representa a custódia de ferramenta por operador e o respetivo estado.
supervisor_resolution	Formaliza a decisão humana sobre pendências ou ocorrências.
supervisor_resolution_assignment	Relaciona resoluções de supervisor com as atribuições abrangidas.
cabinet_event	Regista eventos técnicos relevantes do armário.
audit_log	Guarda registos transversais de ações relevantes para rastreabilidade.

7. Estado atual do desenvolvimento

À data deste relatório intermédio, o trabalho realizado encontra-se concentrado na definição e organização dos ficheiros de especificação e documentação que irão orientar a implementação do *backend*, nomeadamente documentação em Markdown, especificação *OpenAPI* em YAML, descrição dos *workflows*, regras de domínio, modelo de dados essencial e identificação dos *endpoints* principais da API.

Esta documentação técnica permite estabelecer o comportamento esperado do sistema: autenticação do armário, autenticação do operador, abertura de *CabinetAccess*, submissão de *snapshots*, *close* com *operationalResult*, consulta de custódia, *end-of-day-check* e registo de *SupervisorResolution*. A criação e atualização de *ToolAssignment*, bem como as transições de estado entre *ACTIVE*, *RETURNED*, *PENDING_REVIEW* e *RESOLVED*, encontram-se especificadas ao nível da API e do modelo de dados, mas ainda não implementadas em código.

O estado atual é, por isso, adequado para uma entrega intermédia centrada na especificação e no planeamento técnico da API. A fase seguinte deverá transformar esta base documental em implementação funcional, com especial atenção à validação dos *endpoints*, ao tratamento de erros, aos mecanismos de segurança e à criação de testes automáticos.

7.1. Validação e limitações atuais

A validação existente é, por enquanto, documental e centrada na consistência entre os diferentes elementos de especificação. Foram alinhados os principais ficheiros técnicos, incluindo a descrição *OpenAPI*, os *workflows*, as regras de domínio e o modelo de dados essencial.

Os mecanismos de segurança estão especificados ao nível do contrato e da documentação, distinguindo pedidos de armário, operador, supervisor e administração. A validação efetiva desses mecanismos deverá ser implementada e reforçada na fase seguinte.

Na fase de implementação deverão ser definidos com maior detalhe o tratamento das credenciais, a autenticação por perfis e a gestão de PINs. O tratamento de erros e as mensagens encontram-se definidos de forma preliminar e deverão ser consolidados à medida que os serviços e endpoints forem implementados. A cobertura de testes automáticos deverá ser desenvolvida após a criação dos componentes principais da API.

8. Calendarização atualizada e ponto de situação

O planeamento inicial previa uma primeira fase de análise e definição de entidades, seguida da especificação da API e do modelo de dados, e posteriormente da implementação do *backend*. No ponto de situação atual, a análise do funcionamento, a delimitação do âmbito, a especificação da API e o modelo de dados essencial encontram-se mais maduros do que no relatório inicial. A implementação de código não foi ainda iniciada e passa a constituir a prioridade da fase seguinte, mantendo-se enquadrada no período de trabalho previsto para maio.

A seguinte tabela apresenta a calendarização revista. A implementação do *backend* surge concentrada em maio, uma vez que, até à entrega intermédia, o trabalho realizado incide sobretudo na especificação, na documentação e na preparação da implementação.

Tarefa	Mar	Abr	Mai	Jun	Jul
Análise do funcionamento do sistema e definição das entidades principais	X				
Especificação da API e do modelo de dados	X	X			
Documentação <i>OpenAPI</i>		X	X		
Implementação do <i>backend</i>			X		
Implementação do simulador			X		
Testes unitários e de integração			X	X	
Integração, correções e estabilização				X	
Relatório final e preparação da apresentação				X	X

Área	Ponto de situação	Ação seguinte
Análise do funcionamento e entidades	Consolidada para o intermédio	Manter apenas ajustes pontuais decorrentes de feedback.
Especificação da API e modelo de dados	Consolidada ao nível documental	Utilizar <i>OpenAPI</i> como referência e completar exemplos.
Implementação do <i>backend</i>	Por iniciar no código	Implementar os <i>endpoints</i> principais a partir do contrato especificado.
Documentação <i>OpenAPI</i>	Alinhada com o contrato atual	Completar exemplos e rever consistência durante a implementação.
Simulador	Previsto para a fase seguinte, após estabilização da especificação da API	Desenvolver uma versão inicial compatível com a API, orientada à demonstração e validação do fluxo principal.
Testes e validação	Por iniciar sobre código	Adicionar testes de serviços, API e fluxo fim-a-fim após implementação.
Integração e estabilização	Por desenvolver	Consolidar após a implementação dos componentes principais.

9. Próximos passos até ao relatório final

Após a entrega intermédia, o trabalho deverá concentrar-se na implementação do contrato já especificado. A API encontra-se suficientemente estabilizada ao nível documental para servir de base às seguintes fases, pelo que o foco deverá passar para desenvolvimento, validação, testes e demonstração.

As prioridades até ao relatório final são: implementar os *endpoints* principais, reforçar a cobertura de testes, definir mecanismos de segurança mais robustos, estabilizar o tratamento de erros, completar a documentação técnica, preparar exemplos de utilização e iniciar o desenvolvimento de uma versão inicial do simulador, compatível com a API especificada, orientada à demonstração e validação do fluxo principal. A eventual utilização de Docker Compose e outras formas de empacotamento mantém-se como parte da estabilização final do projeto.

- Implementar e validar os serviços e *endpoints* principais da API.
- Definir e implementar mecanismos de autenticação e autorização por perfil.
- Refinar o tratamento de erros e os códigos de resposta da API.
- Produzir evidências de validação fim-a-fim com pedidos e respostas reais.
- Atualizar a documentação técnica com exemplos completos de utilização.
- Desenvolver uma versão inicial do simulador compatível com a API, após estabilização dos endpoints principais.

10. Conclusão

O trabalho desenvolvido desde o relatório inicial permitiu consolidar o projeto em torno de uma abordagem *API-first*, mais adequada ao âmbito da unidade curricular e ao objetivo técnico definido. A delimitação atual evita a dispersão por funcionalidades acessórias e concentra o relatório intermédio na especificação do contrato da API de comunicação para *smart tool cabinets*.

A evolução mais relevante verificou-se na clarificação do fluxo principal, na definição dos requisitos que moldam a API e na estabilização do modelo de dados essencial. O projeto dispõe atualmente de uma base documental e contratual consistente, que permite avançar para a implementação com uma orientação técnica mais clara.

A fase seguinte deverá incidir no desenvolvimento e validação da solução. A prioridade passará pela implementação do *backend*, pela produção de evidências de funcionamento, pela criação de testes, pelo reforço dos mecanismos de segurança, pela preparação de uma versão inicial do simulador e pela estabilização da documentação técnica. Desta forma, o relatório intermédio cumpre a função de apresentar uma evolução clara face ao relatório inicial e de estabelecer uma base técnica concreta para o trabalho até à entrega final.